

Write a Prolog Program to Implement Family Tree Relationships

Aim

To develop a Prolog program that represents a family tree and identifies relationships such as parent, grandparent, brother, and sister.

Objective

- To understand facts and rules in Prolog.
- To represent family relationships using logical predicates.
- To retrieve family relationship information through queries.
- To apply logical reasoning in Artificial Intelligence.

Theory

A **Family Tree** in Prolog is represented using **facts** and **rules**. Facts describe direct relationships such as parent or gender, while rules define indirect relationships such as grandparent, sibling, brother, and sister. Prolog uses logical inference to answer queries by matching facts and applying rules. Family tree representation is a basic example of a **knowledge-based system** in Artificial Intelligence.

Algorithm

Step 1: Define parent relationships as facts.

Step 2: Define gender information.

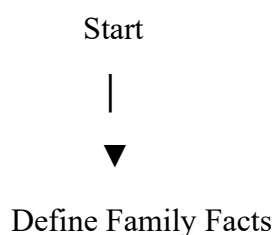
Step 3: Create rules for father, mother, brother, sister, and grandparent.

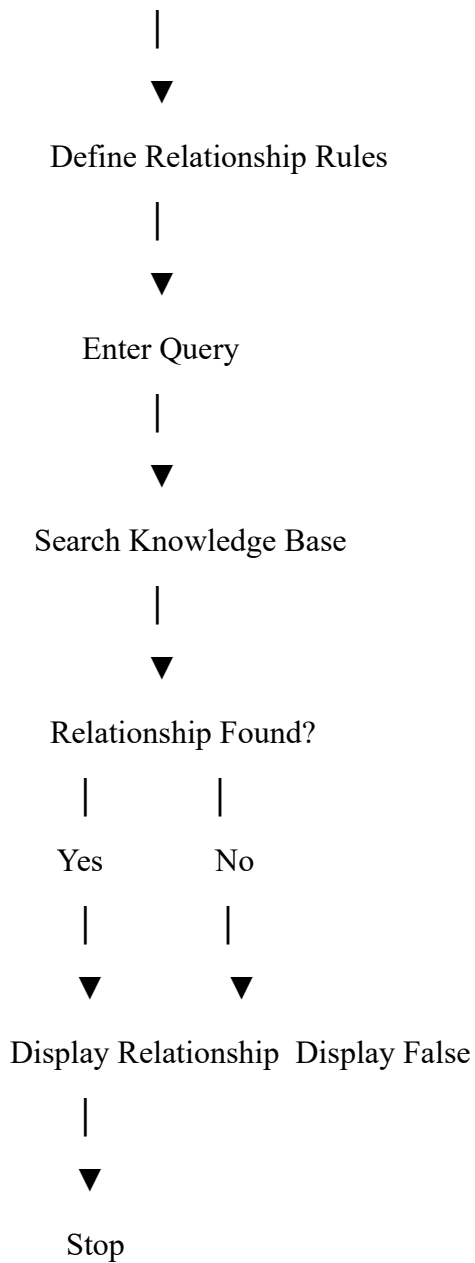
Step 4: Enter a query.

Step 5: Search the knowledge base.

Step 6: Display the required relationship.

Flowchart





Prolog Program

Facts

```
parent = [
    ("ram", "arun"),
    ("ram", "priya"),
    ("sita", "arun"),
    ("sita", "priya")
]
```

```
male = {"ram", "arun"}
```

```
female = {"sita", "priya"}
```

```
# Functions
```

```
def father(x, y):
```

```
    return (x, y) in parent and x in male
```

```
def mother(x, y):
```

```
    return (x, y) in parent and x in female
```

```
def brother(x, y):
```

```
    if x == y:
```

```
        return False
```

```
    for z, child in parent:
```

```
        if child == x:
```

```
            for z2, child2 in parent:
```

```
                if z == z2 and child2 == y and x in male:
```

```
                    return True
```

```
    return False
```

```
def sister(x, y):
```

```
    if x == y:
```

```
        return False
```

```
    for z, child in parent:
```

```
        if child == x:
```

```
            for z2, child2 in parent:
```

```
                if z == z2 and child2 == y and x in female:
```

```
                    return True
```

```
    return False
```

```
def grandparent(x, z):
```

```
    for p, y in parent:
```

```
        if p == x:
```

```
            if (y, z) in parent:
```

```
                return True
```

```
    return False
```

```
# Sample queries
```

```
print("father(ram, arun):", father("ram", "arun"))
```

```
print("mother(sita, priya):", mother("sita", "priya"))
```

```
print("brother(arun, priya):", brother("arun", "priya"))
```

```
print("sister(priya, arun):", sister("priya", "arun"))
```

```
print("grandparent(ram, X):", grandparent("ram", "arun"))
```

Sample Output

```
?- father(ram, arun).
```

```
true.
```

```
?- sister(priya, arun).
```

```
true.
```

```
?- grandparent(X, arun).
```

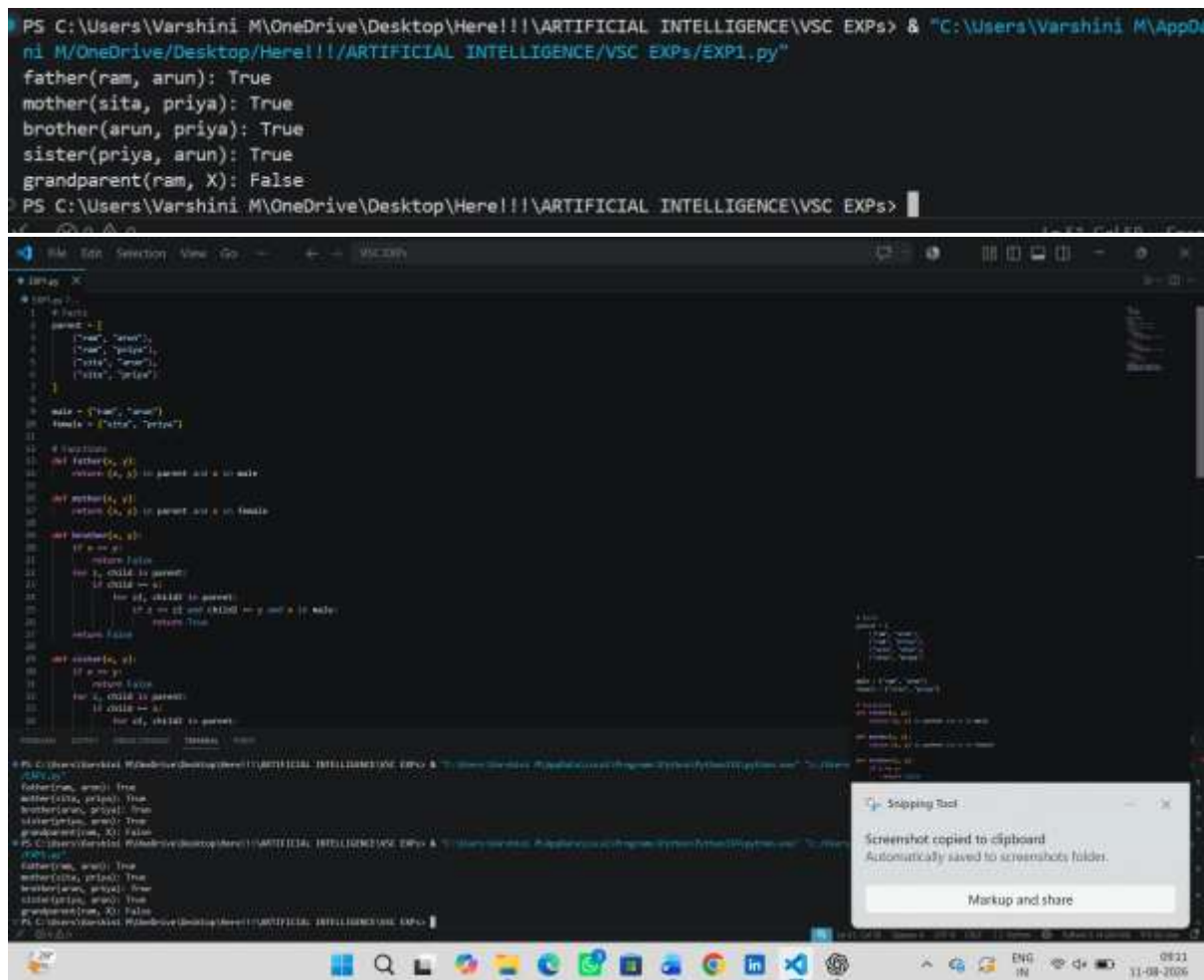
```
false.
```

Result

The Prolog program successfully represented family relationships and answered logical queries.

Conclusion

This experiment demonstrates how Prolog uses facts and rules to represent family relationships and perform logical reasoning.



The screenshot displays a Windows environment with a terminal window and a VS Code editor. The terminal window shows the execution of a Prolog program, and the VS Code editor shows the source code of the program.

Terminal Output:

```
PS C:\Users\Varshini M\OneDrive\Desktop\Here!!!\ARTIFICIAL INTELLIGENCE\VSC EXPs> "C:\Users\Varshini M\AppData\Local\Microsoft\Windows\Terminal\Profiles\WindowsTerminal\code.exe" "C:\Users\Varshini M\OneDrive\Desktop\Here!!!\ARTIFICIAL INTELLIGENCE\VSC EXPs\EXP1.py"
father(ram, arun): True
mother(sita, priya): True
brother(arun, priya): True
sister(priya, arun): True
grandparent(ram, X): False
PS C:\Users\Varshini M\OneDrive\Desktop\Here!!!\ARTIFICIAL INTELLIGENCE\VSC EXPs>
```

VS Code Editor Content:

```
1 # Facts
2 parent :-
3     ['ram', 'sita'],
4     ['ram', 'priya'],
5     ['sita', 'arun'],
6     ['sita', 'priya']
7
8
9 male = ['ram', 'arun']
10 female = ['sita', 'priya']
11
12 # Rules
13 def father(x, y):
14     return (x, y) in parent and x is male
15
16 def mother(x, y):
17     return (x, y) in parent and x is female
18
19 def brother(x, y):
20     if x == y:
21         return False
22     for i, child in parent:
23         if child == x:
24             for aj, child2 in parent:
25                 if x == aj and ARJID == y and x is male:
26                     return True
27     return False
28
29 def sister(x, y):
30     if x == y:
31         return False
32     for i, child in parent:
33         if child == x:
34             for aj, child2 in parent:
```

Terminal Output (continued):

```
PS C:\Users\Varshini M\OneDrive\Desktop\Here!!!\ARTIFICIAL INTELLIGENCE\VSC EXPs> "C:\Users\Varshini M\AppData\Local\Microsoft\Windows\Terminal\Profiles\WindowsTerminal\code.exe" "C:\Users\Varshini M\OneDrive\Desktop\Here!!!\ARTIFICIAL INTELLIGENCE\VSC EXPs\EXP1.py"
father(ram, arun): True
mother(sita, priya): True
brother(arun, priya): True
sister(priya, arun): True
grandparent(ram, X): False
PS C:\Users\Varshini M\OneDrive\Desktop\Here!!!\ARTIFICIAL INTELLIGENCE\VSC EXPs>
```

A "Snipping Tool" notification is visible in the bottom right corner, indicating that a screenshot was copied to the clipboard and saved to the screenshots folder.